

Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра №12 «Компьютерные системы и технологии»



ОТЧЕТ

О выполнении лабораторной работы №3 «Работа с массивами данных»

Студент: Сейтназаров Д.Н.

Группа: Б24-703

Преподаватель: Храпов А.С

Москва – 2024

1. Формулировка индивидуального задания

Вариант №36. В исходной последовательности целых чисел найти те, которые состоят из одинаковых цифр. Сформировать из данных чисел новую последовательность, удалив их из исходной.

2. Описание использованных типов данных

При выполнении данной лабораторной работы использовался встроенный тип данных `int`, предназначенный для работы с целыми числами.

3. Описание использованного алгоритма

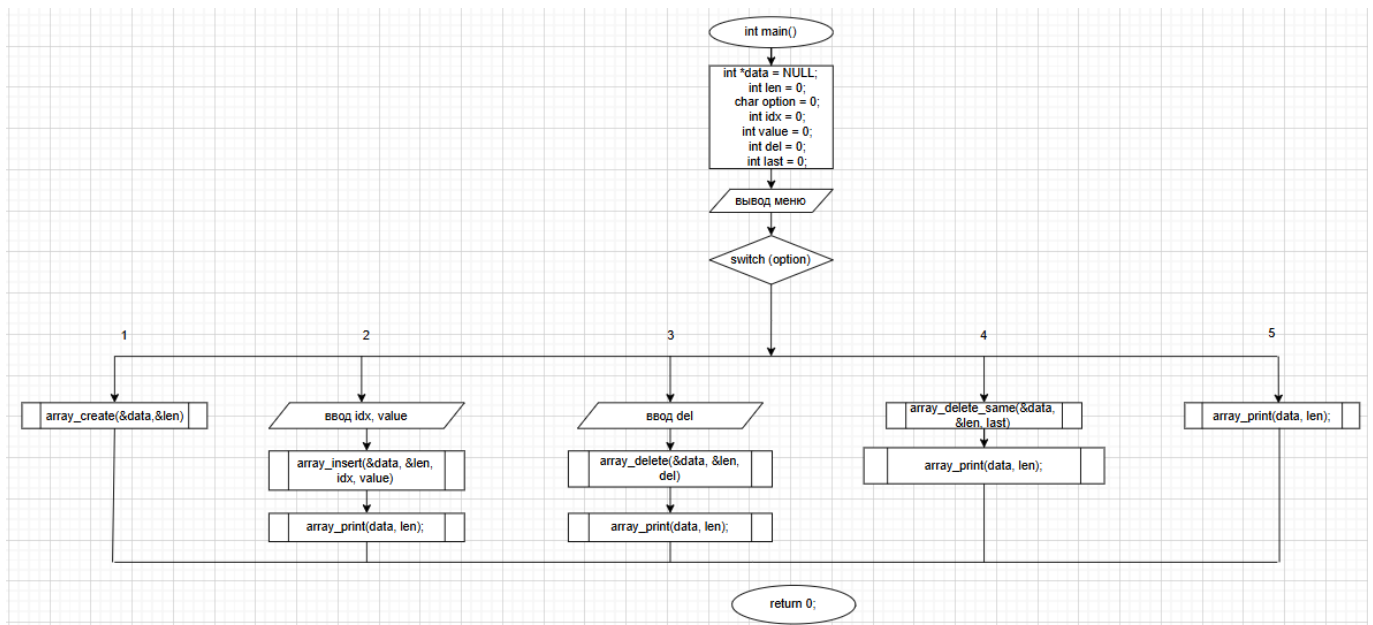


Рис. 1: Блок-схема алгоритма работы функции `main()`

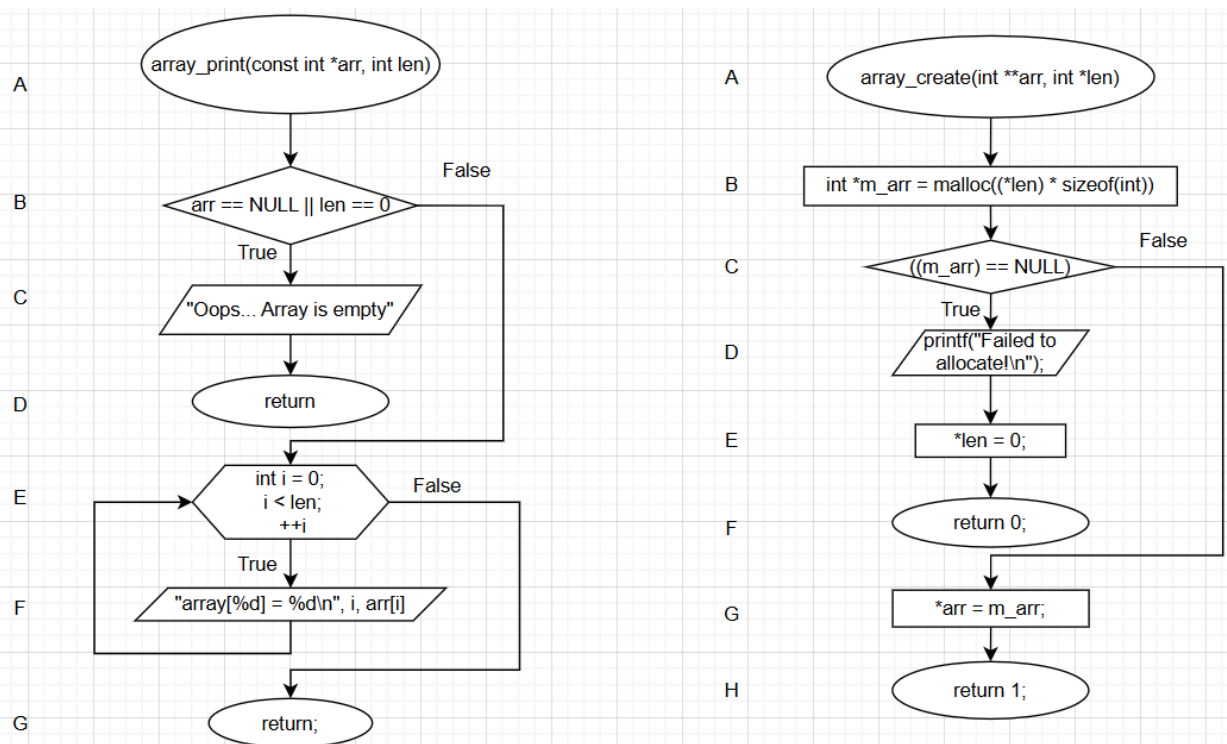


Рис. 2: Блок-схема алгоритма работы функций `array_create()` & `array print()`

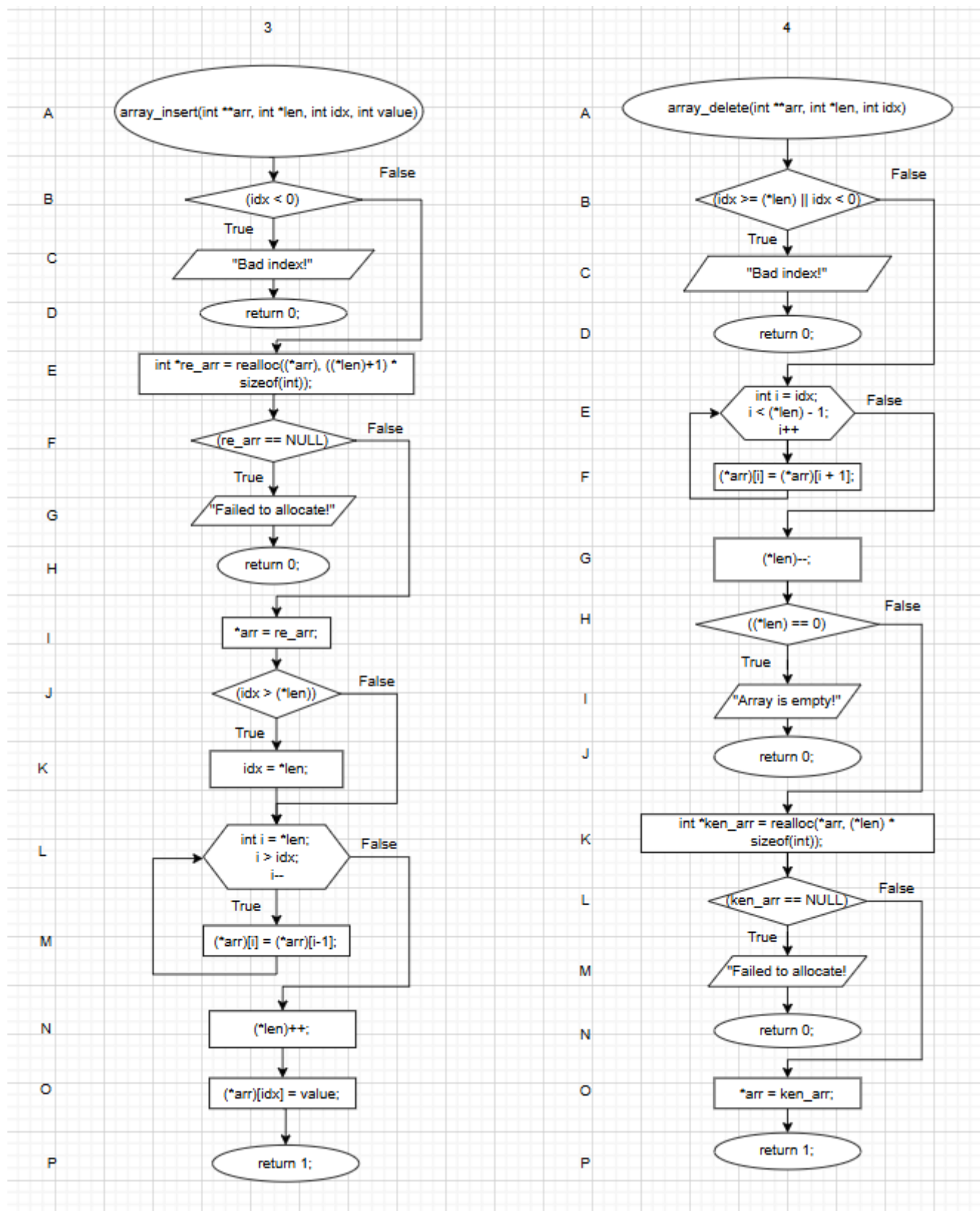


Рис. 3: Блок-схема алгоритма работы функций `array_insert()` & `array_delete()`

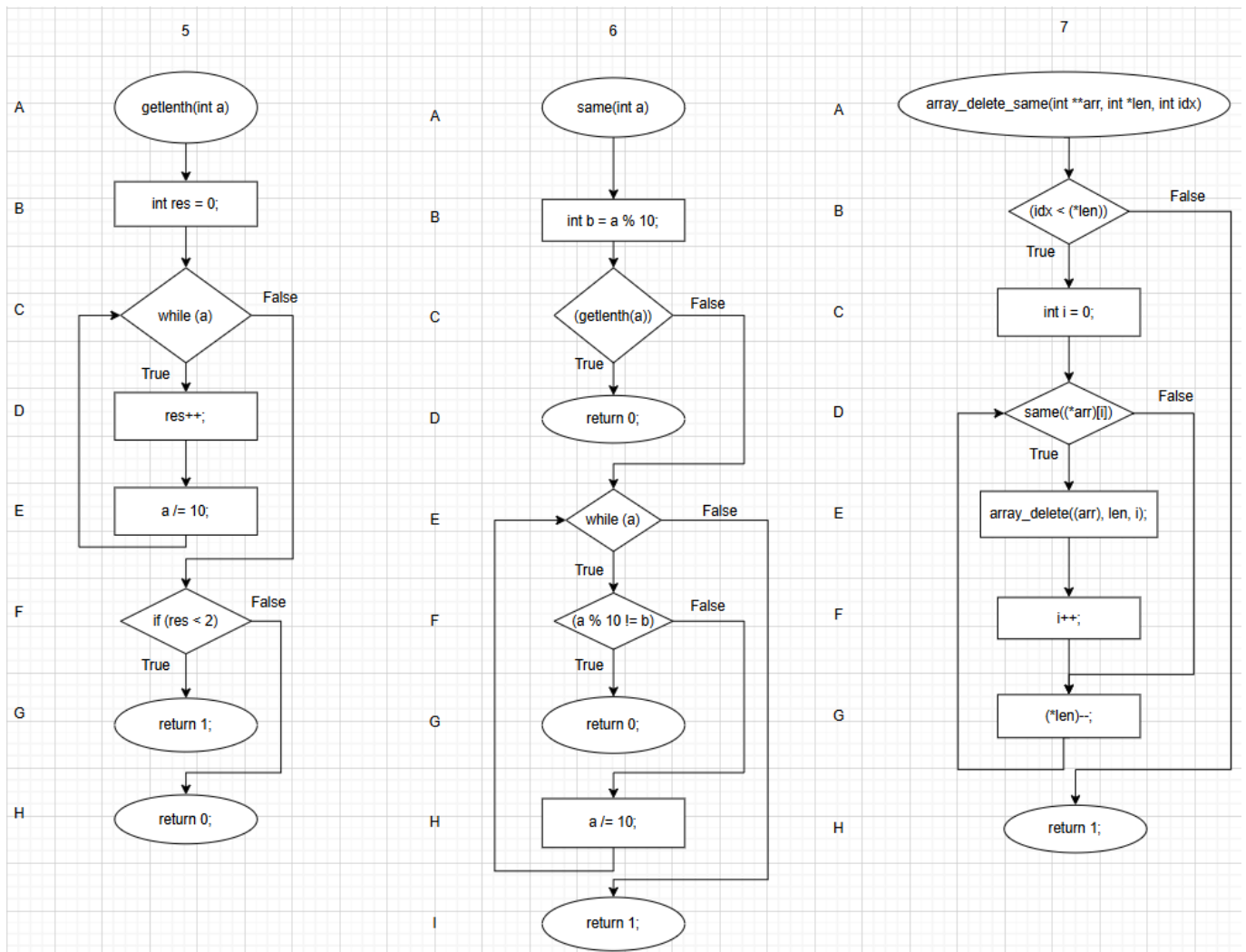


Рис. 4: Блок-схема алгоритма работы функций `getlenth()` & `same()` & `array_detele_same()`

4. Исходные коды разработанных программ

Листинг 1: Исходные коды программы main (файл: main.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "func.h"
4
5 int main() {
6     int *data = NULL;
7     int len = 0;
8     char option = 0;
9     int idx = 0;
10    int value = 0;
11    int del = 0;
12    while (option != '0'){
13        printf("\n");
14        printf("***** MENU *****\n");
15        printf("1. Create an array\n");
16        printf("2. Insert element\n");
17        printf("3. Delete element\n");
18        printf("4. Delete elements with same values\n");
19        printf("5. Print array\n");
20        printf("0. Exit\n");
21        printf("\n");
22        int r = scanf(" %c", &option);
23        if (r == EOF) {
24            free(data);
25            return 0;
26        }
27        switch (option){
28            case '1':
29                if (len > 0){
30                    len = 0;
31                    free(data);
32                }
33                if (array_create(&data,&len)){
34                    printf("Success!\n");
35                }
36                else{
37                    printf("Failed to allocate!\n");
38                }
39                array_print(data, len);
40                break;
41            case '2':
42                printf("Enter index : \n");
43                if (scanf("%d", &idx) < 0){
44                    printf("Bad index!\nTry again with correct index!\n");
45                    break;
46                }
47                printf("Enter value : \n");
48                scanf("%d", &value);
49                if (array_insert(&data,&len,idx,value)){
50                    printf("Success!\n");
51                }
52                else {
53                    printf("Failed to allocate!\n");
54                }
55                array_print(data, len);
```

```

56         break;
57     case '3':
58         printf("Enter index : \n");
59         if (scanf("%d", &del) < 0){
60             printf("Bad index!\nTry again with correct index!\n");
61             break;
62         }
63         if (array_delete(&data, &len, del)){
64             printf("Success!\n");
65         }
66         else {
67             printf("Failed to allocate!\n");
68         }
69         array_print(data, len);
70         break;
71     case '4':
72         printf("Delete elements with same numbers!\n");
73         array_delete_same(&data, &len);
74         array_print(data, len);
75         break;
76     case '5':
77         array_print(data, len);
78         break;
79     case '0':
80         printf("Exit\n");
81         free(data);
82         return 0;
83     default:
84         printf("Wrong input!\n");
85         break;
86     }
87 }
88 free(data);
89 return 0;
90 }

```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #include "func.h"
5
6 void array_print(const int *arr, int len) {
7     if (arr == NULL || len == 0) {
8         printf("Array is empty\n");
9         return;
10    }
11    for (int i = 0; i < len; ++i) {
12        printf("array[%d] = %d\n", i, arr[i]);
13    }
14 }
15
16 int array_create(int** arr, int* len){
17     int *m_arr = malloc((*len) * sizeof(int));
18
19     if (m_arr == NULL){
20         (*len) = 0;
21         return 0;
22     }

```

```

23
24     (*arr) = m_arr;
25     return 1;
26 }
27
28 int array_insert(int **arr, int *len, int idx, int value){
29     int (*re_arr) = realloc((*arr), ((*len)+1) * sizeof(int));
30     if (re_arr == NULL){
31         return 0;
32     }
33     (*arr) = re_arr;
34
35     if (idx > (*len)){
36         idx = *len;
37     }
38
39     for (int i = *len; i > idx; i--) {
40         (*arr)[i] = (*arr)[i-1];
41     }
42
43     (*len)++;
44     (*arr)[idx] = value;
45     return 1;
46 }
47
48 int array_delete(int **arr, int *len, int idx){
49     for (int i = idx; i < (*len) - 1; i++) {
50         (*arr)[i] = (*arr)[i + 1];
51     }
52
53     (*len)--;
54     if ((*len) == 0){
55         return 1;
56     }
57
58     int *ken_arr = realloc(*arr, (*len) * sizeof(int));
59     if (ken_arr == NULL){
60         return 0;
61     }
62
63     (*arr) = ken_arr;
64     return 1;
65 }
66
67 int getlenth(int a){
68     int res = 0;
69
70     while (a){
71         res++;
72         a /= 10;
73     }
74     if (res < 2){
75         return 1; // Число меньше 10.
76     }
77     else {
78         return 0;
79     }
80 }
81

```



```

82 int same(int a){
83     int b = a % 10;
84     if (getlenth(a)){
85         return 0; // Если getlenth = 1, то число меньше 10.
86     }
87     while (a) {
88         if (a % 10 != b) {
89             return 0;
90         }
91         a /= 10;
92     }
93     return 1;
94 }
95
96 int array_delete_same(int **arr, int *len){
97     for (int i = 0; i < (*len); ){
98         if (same((*arr)[i])){
99             array_delete(arr, len, i);
100         }
101         else{
102             i++;
103         }
104     }
105     return 1;
106 }

1 #ifndef FUNC_H_INCLUDED
2 #define FUNC_H_INCLUDED
3
4 void array_print(const int *arr, int len);
5 int array_create(int **arr, int *len);
6 int array_insert(int **arr, int *len, int idx, int value);
7 int array_delete(int **arr, int *len, int idx);
8 int array_delete_same(int **arr, int *len);
9 int getlenth(int a);
10 int same(int a);
11
12 #endif // FUNC_H_INCLUDED

```

Листинг 1: Исходные коды программ main.c, func.c, func.h (Файлы с теми же именами)

5. Описание тестовых примеров

Таблица 1: Создание массива (array_create)

option	len	idx	value	array_print
1	0	-	-	Array Empty

Таблица 2: Добавление первого элемента (array_insert)

option	len	idx	value	array_print
2	1	0	5	array - [5]

Таблица 3: Добавление второго элемента (array_insert)

option	len	idx	value	array_print
2	2	1	44	array - [5,44]

Таблица 4: Удаление одинаковых элементов (array_delete_same)

option	len	idx	value	array_print
4	1	-	-	array - [5]

Таблица 5: Вывод массива (array_print)

option	len	idx	value	array_print
5	1	-	-	array - [5]

6. Скриншоты

```
[seitnazarov.dn@unix lab3]$ [seitnazarov.dn@unix lab3]$ ls
func.c  func.h  main.c
[seitnazarov.dn@unix lab3]$ gcc main.c func.c -g -o file
[seitnazarov.dn@unix lab3]$ valgrind ./file
==31617== Memcheck, a memory error detector
==31617== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==31617== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==31617== Command: ./file
==31617==

***** MENU *****
1. Create an array
2. Insert element
3. Delete element
4. Delete elements with same values
5. Print array
0. Exit

1
Success!
Oops... Array is empty

***** MENU *****
1. Create an array
2. Insert element
3. Delete element
4. Delete elements with same values
5. Print array
0. Exit

2
Enter index :
33
Enter value :
2
Success!
array[0] = 2

***** MENU *****
1. Create an array
2. Insert element
3. Delete element
4. Delete elements with same values
5. Print array
0. Exit
```

```

3
Enter index :
0
Array is empty!
Success!
Oops... Array is empty

**** MENU ****
1. Create an array
2. Insert element
3. Delete element
4. Delete elements with same values
5. Print array
0. Exit

==31617==
==31617== HEAP SUMMARY:
==31617==    in use at exit: 0 bytes in 0 blocks
==31617==   total heap usage: 4 allocs, 4 frees, 2,052 bytes allocated
==31617==
==31617== All heap blocks were freed -- no leaks are possible
==31617==
==31617== For lists of detected and suppressed errors, rerun with: -s
==31617== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[seitnazarov.dn@unix lab3]$

```

Рис. 3: Сборка и запуск программ main.c и func.c

7. Выводы

В ходе выполнения данной работы на примере программы, выполняющей сложение целых чисел, были рассмотрены базовые принципы работы построения программ на языке C и обработки целых чисел:

В ходе выполнения данной работы на примере программы, которая позволяет добавлять/удалять элементы, были рассмотрены следующие принципы работы с массивами:

1. Выделение/очистка памяти.
2. Работа с указателем на указатель.
3. Проверка утечки памяти с помощью Valgrind.
4. Работа с оператором switch, который позволил сделать меню для программы.